Name = Harsh Kumar gola

CU24250165

BTech CSE 3rd Year SECTION - "B"

Roll No. :- 23

==========

Experiment Name

Exploratory Data Analysis (EDA) on a Real-World Business

Dataset using Python

==========

Aim:

To perform Exploratory Data Analysis (EDA) on a real-world dataset using Python in order to understand the dataset's structure, identify trends, detect anomalies, analyze feature relationships, and generate meaningful business insights through descriptive statistics and visualizations.

Step 1: Import Required Libraries

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from tkinter import Tk, filedialog
```

step 2: Select / Upload Dataset from Computer

```
from google.colab import files

print("Please upload your cleaned business dataset.")
uploaded = files.upload()

if not uploaded:
    print("No dataset selected.")
    file_path = None
else:
    # Get the name of the first uploaded file
    file_path = next(iter(uploaded.keys()))
    print(f"File '{file_path}' uploaded successfully.")
```

```
Please upload your cleaned business dataset.
 ChooseFiles Nofilechosen    Uploadwidgetisonly available when the cell has been executed in the current browser session. Please rerun this cell to enable.
Saving Mall_Customers.csv to Mall_Customers.csv
File 'Mall_Customers.csv' uploaded successfully.
```

Step 3: Load Dataset

```
if file_path.lower().endswith(".csv"):
    df = pd.read_csv(file_path)

elif file_path.lower().endswith((".xlsx", ".xls")):
    df = pd.read_excel(file_path)

else:
    print("Unsupported file format.")
    # Instead of exiting, we'll set df to None or handle as appropriate
    df = None # Or raise an error if strict file type is required


if df is not None:
    print("Dataset loaded successfully.")
    print("Selected Dataset:", file_path)
else:
    print("Failed to load dataset.")
```

```
Dataset loaded successfully.
Selected Dataset: Mall_Customers.csv
```

Step 4: Display First Five Records

```
print("\n========== FIRST FIVE RECORDS ==========")

print(df.head())
```

```
========== FIRST FIVE RECORDS ==========
   CustomerID    Genre Age   Annual Income (k$)  Spending Score (1-100)
0  1             Male    19                  15                      39
1  2             Male    21                  15                      81
2  3 Female 4 Female 5  20                  16                       6
3  Female               23                  16                      77
4                       31                  17                      40
```

Step 5: Display Dataset Dimensions

```
print("\n========== DATASET DIMENSIONS ==========")

print("Number of Rows:", df.shape[0])
print("Number of Columns:", df.shape[1])
```

```
========== DATASET DIMENSIONS ==========
Number of Rows: 200
Number of Columns: 5
```

step 6: Display Column Names

```
print("\n========== COLUMN NAMES ==========")

print(df.columns.tolist())
```

```
========== COLUMN NAMES ==========
['CustomerID', 'Genre', 'Age', 'Annual Income (k$)', 'Spending Score (1-100)']
```

Step 7: Display Data Types

```
print("\n========== DATA TYPES ==========")

print(df.dtypes)
```

```
========== DATA TYPES ==========
CustomerID                int64
```

```
Genre                    object
Age                       int64
Annual Income (k$)        int64
Spending Score (1-100)    int64
dtype: object
```

Step 8: Display Dataset Information

```
print("\n========== DATASET INFORMATION ==========")

df.info()
```

```
========== DATASET INFORMATION ==========
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 200 entries, 0 to 199
Data columns (total 5 columns):
 #   Column                  Non-Null Count  Dtype
---  ------                  --------------  -----
 0   CustomerID              200   non-null  int64
1    Genre                   200   non-null  object
2    Age                     200   non-null  int64
3    Annual Income (k$)      200   non-null  int64
4    Spending Score (1-100)  200 non-null    int64
dtypes: int64(4), object(1)
memory usage: 7.9+ KB
```

Step 9: Statistical Summary

```
print("\n========== STATISTICAL SUMMARY ==========")

print(df.describe(include="all"))
```

```
========== STATISTICAL SUMMARY ==========
        CustomerID   Genre          Age  Annual Income (k$) \
count   200.000000     200  200.000000          200.000000
unique  NaN              2         NaN                 NaN
top     NaN Female               NaN                 NaN
freq                             NaN                 NaN
mean            NaN    112   38.850000           60.560000
std   min100.500000    NaN   13.969007           26.264721
25%   50% 57.879185    NaN   18.000000           15.000000
75% max     1.000000   NaN   28.750000           41.500000
          50.750000    NaN   36.000000           61.500000
        100.500000     NaN   49.000000           78.000000
        150.250000     NaN   70.000000          137.000000
        200.000000     NaN


         Spending Score (1-100)
count                200.000000
unique                      NaN
top                         NaN
freq                        NaN
mean                  50.200000
std   min             25.823522
25%   50%              1.000000
75% max               34.750000
                      50.000000
                      73.000000
                      99.000000
```

Step 10: Missing Value Analysis

```
print("\n========== MISSING VALUES ==========")

missing_values = df.isnull().sum()

print(missing_values)

print("\nTotal Missing Values:",
      df.isnull().sum().sum())
```

```
========== MISSING VALUES ==========
CustomerID              0
Genre                   0
Age                     0
Annual Income (k$)      0
Spending Score (1-100)  0
dtype: int64


Total Missing Values: 0
```

Step 11: Duplicate Analysis

```
print("\n========== DUPLICATE RECORDS ==========")

duplicate_count = df.duplicated().sum()

print("Number of Duplicate Records:",
      duplicate_count)
```

```
========== DUPLICATE RECORDS ==========
Number of Duplicate Records: 0
```

Step 12: Identify Numerical and Categorical Columns

```
numerical_columns = df.select_dtypes(
    include=np.number
).columns.tolist()

categorical_columns = df.select_dtypes(
    exclude=np.number
).columns.tolist()

print("\n========== NUMERICAL FEATURES ==========")

print(numerical_columns)

print("\n========== CATEGORICAL FEATURES ==========")

print(categorical_columns)
```

```
========== NUMERICAL FEATURES ==========
['CustomerID', 'Age', 'Annual Income (k$)', 'Spending Score (1-100)']

========== CATEGORICAL FEATURES ==========
['Genre']
```

Step 13: Value Counts for Categorical Variables

```
print("\n========== CATEGORICAL VALUE COUNTS ==========")

for column in categorical_columns:

    print("\nColumn:", column)

    print(
        df[column]
        .value_counts()
        .head(10)
    )
```

```
========== CATEGORICAL VALUE COUNTS ==========

Column: Genre
Genre
Female    112
Male       88
Name: count, dtype: int64
```

Step 14: Univariate Analysis - Histograms

```
print("\n========== UNIVARIATE ANALYSIS ==========")

if len(numerical_columns) > 0:

    for column in numerical_columns:

        plt.figure(figsize=(8, 5))

        sns.histplot(
            data=df,
            x=column,
            kde=True
        )

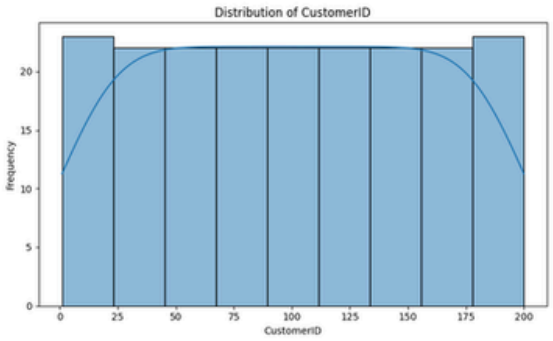
        plt.title(
            "Distribution of " + column
        )

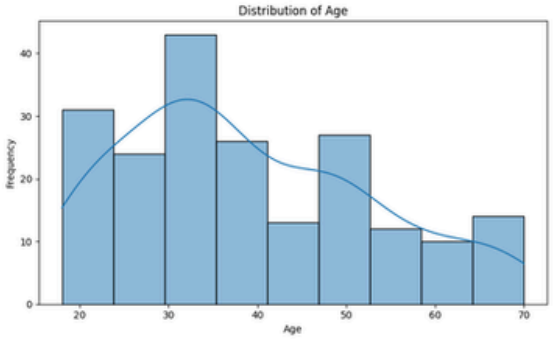
        plt.xlabel(column)
        plt.ylabel("Frequency")

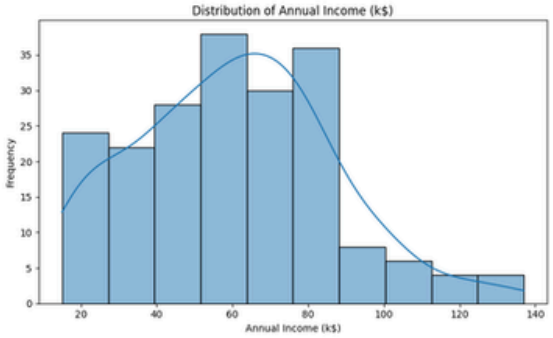
        plt.tight_layout()
        plt.show()

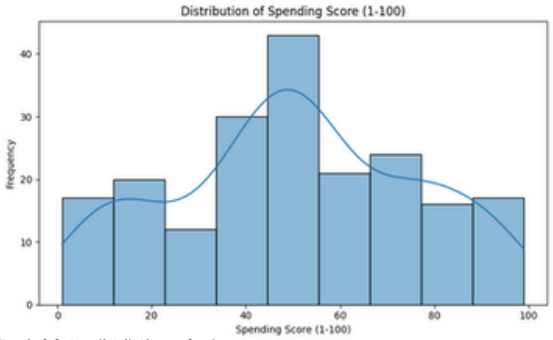
print("Numerical feature distributions analyzed.")
```

```
========== UNIVARIATE ANALYSIS ==========
```










```
Numerical feature distributions analyzed.
```

Step 15: Categorical Variable Analysis

```
if len(categorical_columns) > 0:

    # Select first few categorical columns
    selected_categories = categorical_columns[:4]

    for column in selected_categories:

        # Limit categories for better visualization
        top_categories = (
        df[column]
        .value_counts()
        .head(10)
        )


        plt.figure(figsize=(10, 5))
```

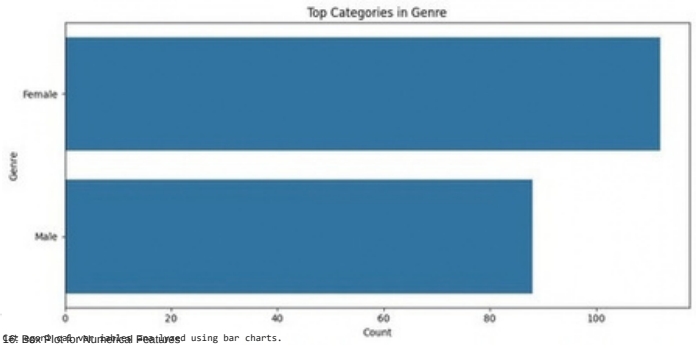
```
        sns.barplot(
            x=top_categories.values,
            y=top_categories.index
        )

        plt.title(
            "Top Categories in " + column
        )

        plt.xlabel("Count")
        plt.ylabel(column)

        plt.tight_layout()
        plt.show()

print("Categorical variables analyzed using bar charts.")
```




```
Categorical variables analyzed using bar charts.
```

Step 16: Box Plot for Numerical Features

```
if len(numerical_columns) > 0:

    plt.figure(figsize=(12, 6))

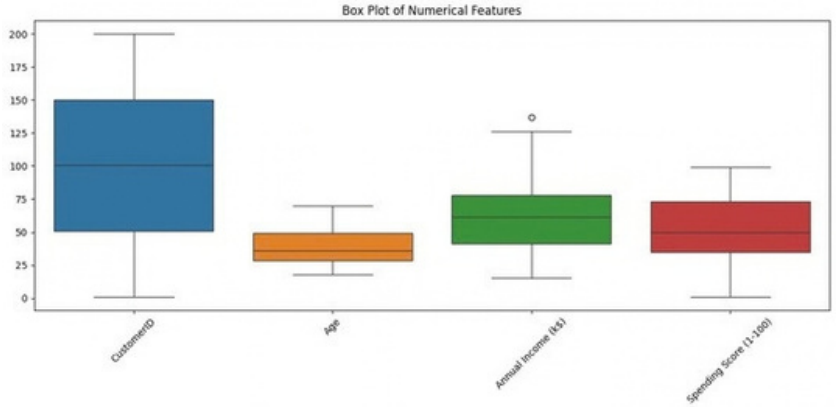
    sns.boxplot(
        data=df[numerical_columns]
    )

    plt.title(
        "Box Plot of Numerical Features"
    )

    plt.xticks(rotation=45)

    plt.tight_layout()
    plt.show()

print("Box plot generated successfully.")
```




```
Box plot generated successfully.
```

Step 17: Correlation Matrix

```
if len(numerical_columns) >= 2:

    correlation_matrix = df[
        numerical_columns
    ].corr()

    print("\n========== CORRELATION MATRIX ==========")

    print(correlation_matrix)
```

```
========== CORRELATION MATRIX ==========
                        CustomerID       Age Annual Income (k$)   \
CustomerID                1.000000 -0.026763           0.977548
Age                      -0.026763  1.000000          -0.012398
Annual Income (k$)        0.977548 -0.012398           1.000000
Spending Score (1-100)    0.013835 -0.327227           0.009903

                        Spending Score (1-100)
CustomerID                            0.013835
Age                                  -0.327227
Annual Income (k$)                    0.009903
Spending Score (1-100)                1.000000
```

Step 18: Correlation Heatmap

```
if len(numerical_columns) >= 2:

    plt.figure(
        figsize=(12, 8)
    )

    sns.heatmap(
        correlation_matrix,
        annot=True,
        cmap="coolwarm",
        fmt=".2f",
        linewidths=0.5
    )

    plt.title(
        "Correlation Heatmap"
    )

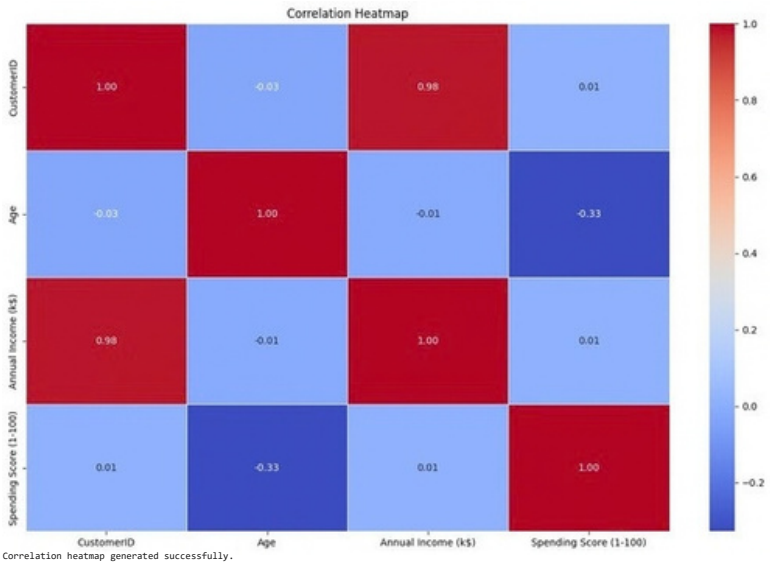
    plt.tight_layout()
```

```
    plt.show()

    print(
        "Correlation heatmap generated successfully."
    )

else:

    print(
        "\nNot enough numerical features "
        "for correlation analysis."
    )
```




```
Correlation heatmap generated successfully.
```

Step 19: Bivariate Analysis

```
print("\n========== BIVARIATE ANALYSIS ==========")

if len(numerical_columns) >= 2:

    x_column = numerical_columns[0]
    y_column = numerical_columns[1]

    plt.figure(figsize=(8, 6))

    sns.scatterplot(
    data=df,
    x=x_column,
    y=y_column
    )


    plt.title(
        x_column + " vs " + y_column
    )

    plt.xlabel(x_column)
    plt.ylabel(y_column)

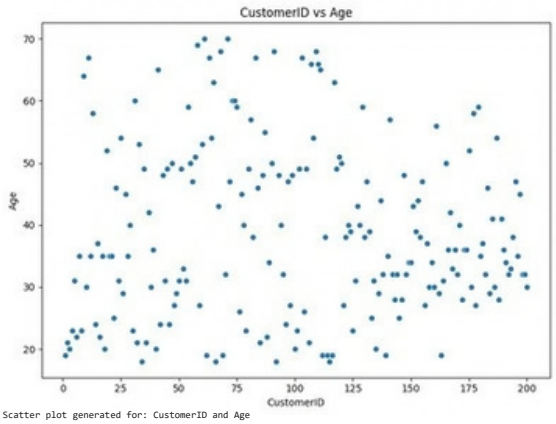
    plt.tight_layout()
    plt.show()

    print(
        "Scatter plot generated for:",
        x_column,
        "and",
        y_column
    )

else:

    print(
        "Not enough numerical columns "
        "for scatter plot."
    )
```

```
========== BIVARIATE ANALYSIS ==========
```




```
Scatter plot generated for: CustomerID and Age
```

Step 20: Multivariate Analysis

```
print("\n========== MULTIVARIATE ANALYSIS ==========")

if len(numerical_columns) >= 3:

    sns.pairplot(
        df[numerical_columns[:5]].dropna()
    )

    plt.show()

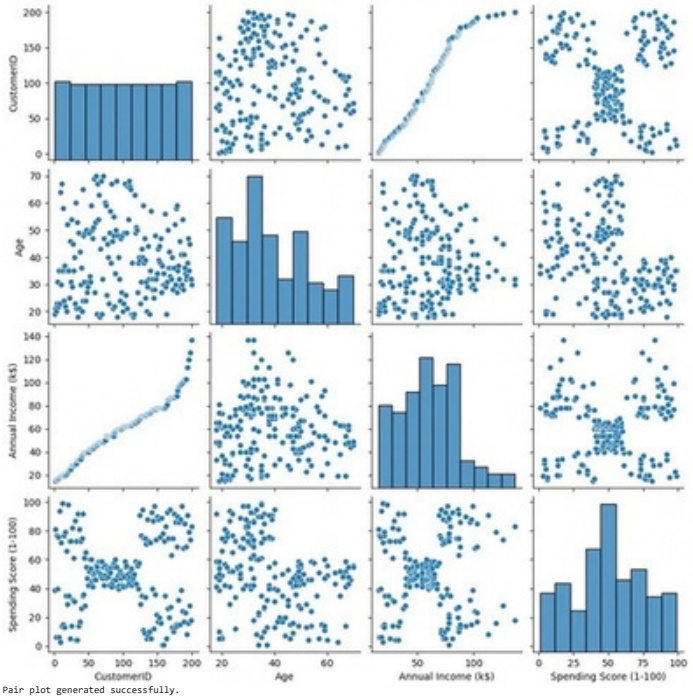
    print(
        "Pair plot generated successfully."
```

```
    )

else:

    print(
        "Not enough numerical features "
        "for multivariate analysis."
    )
```

```
========== MULTIVARIATE ANALYSIS ==========
```




```
Pair plot generated successfully.
```

Step 21: Mean, Median and Standard Deviation

```
print("\n========== DESCRIPTIVE STATISTICS ==========")

if len(numerical_columns) > 0:

    for column in numerical_columns:

        print("\nFeature:", column)

        print(
            "Mean:",
            round(df[column].mean(), 2)
        )

        print(
            "Median:",
            round(df[column].median(), 2)
        )

        print(
            "Standard Deviation:",
            round(df[column].std(), 2)
        )

        print(
            "Minimum:",
            df[column].min()
        )

        print(
            "Maximum:",
            df[column].max()
        )
```

```
========== DESCRIPTIVE STATISTICS ==========

Feature: CustomerID
Mean: 100.5
Median: 100.5
Standard Deviation: 57.88
Minimum: 1
Maximum: 200


Feature: Age
Mean: 38.85
Median: 36.0
Standard  Deviation:  13.97
Minimum: 18
Maximum: 70


Feature: Annual Income (k$)
Mean: 60.56
Median: 61.5
Standard Deviation: 26.26
Minimum: 15
Maximum: 137


Feature: Spending Score (1-100)
Mean: 50.2
Median: 50.0
Standard Deviation: 25.82
Minimum: 1
Maximum: 99
```

Step 22: Detect Highly Correlated Features

```
if len(numerical_columns) >= 2:

    correlation_matrix = df[
        numerical_columns
    ].corr()

    print(
        "\n========== HIGH CORRELATIONS =========="
    )

    found_correlation = False

    for i in range(
        len(correlation_matrix.columns)
    ):
```

```
        for j in range(i+ 1,
                       len(correlation_matrix.columns)):

            correlation_value = (
                correlation_matrix.iloc[i, j]
            )

            if abs(correlation_value) >= 0.70:

                print(
                    correlation_matrix.columns[i],
                    "<->",
                    correlation_matrix.columns[j],
                    ":",
                    round(correlation_value, 2)
                )

                found_correlation = True

    if not found_correlation:

        print(
            "No strongcorrelation "
            "(absolutecorrelation >= 0.70) found."
        )
```

```
========== HIGH CORRELATIONS ==========
CustomerID <-> Annual Income (k$) : 0.98
```

Step 23: Business Insights

```
print("\n========== BUSINESS INSIGHTS ==========")

print(
    "1. Numerical distributions were analyzed "
    "using histograms and density plots."
)

print(
    "2. Frequently occurring categories were "
    "identified using value counts and bar charts."
)

print(
    "3. Box plots were used to identify "
    "potential outliers."
)

print(
    "4. Correlation analysis was used to "
    "identify relationships between numerical features."
)

print(
    "5. Scatter plots were used to analyze "
    "relationships between pairs of variables."
)

print(
    "6. Multivariate analysis was used to "
    "observe relationships among multiple features."
)
```

```
========== BUSINESS INSIGHTS ==========
1. Numerical distributions were analyzed using histograms and density plots.
2. Frequently occurring categories were identified using value counts and bar charts.
3. Box plots were used to identify potential outliers.
4. Correlation analysis was used to identify relationships between numerical features.
5. Scatter plots were used to analyze relationships between pairs of variables.
6. Multivariate analysis was used to observe relationships among multiple features.
```

Step 24: Final Output

```
print(
    "\n========== EXPLORATORY DATA ANALYSIS =========="
)

print("Dataset       Loading:       Completed")
print("Dataset      Inspection:     Completed")
print("Statistical     Analysis:    Completed")
print("Missing   Value   Analysis:  Completed")
print("Duplicate      Analysis:     Completed")
print("Univariate     Analysis:     Completed")
print("Categorical     Analysis:    Completed")
print("Correlation     Analysis:    Completed")
print("Heatmap      Generation:     Completed")
print("Box     Plot     Analysis:   Completed")
print("Scatter    Plot    Analysis: Completed")
print("Multivariate     Analysis:   Completed")
print("Business Insight Generation: Completed")

print("\nExperiment Completed Successfully.")
```

```
========== EXPLORATORY DATA ANALYSIS ==========
Dataset Loading: Completed
Dataset Inspection: Completed
Statistical Analysis: Completed
Missing Value Analysis: Completed
Duplicate Analysis: Completed
Univariate Analysis: Completed
Categorical Analysis: Completed
Correlation Analysis: Completed
Heatmap Generation: Completed
Box Plot Analysis: Completed
Scatter Plot Analysis: Completed
Multivariate Analysis: Completed
Business Insight Generation: Completed


Experiment Completed Successfully.
```

EXPERIMENT 2 - ANSWERS TO QUESTIONS

QUESTION 1: What is Exploratory Data Analysis (EDA), and why is it performed before machine learning?

Exploratory Data Analysis (EDA) is the process of examining, summarizing and visualizing a dataset to understand its structure, patterns, relationships and anomalies.

EDA is performed before machine learning because it helps identify:

Missing values. Duplicate records. Outliers. Data distributions. Relationships between features. Skewness. Class imbalance.

EDA helps analysts understand the data before selecting features, preprocessing techniques and machine learning models.

QUESTION 2: Differentiate between univariate, bivariate, and multivariate analysis with suitable examples.

Univariate Analysis:

Analysis of a single variable.

Example: Studying the distribution of employee age using a histogram.

Bivariate Analysis:

Analysis of two variables to understand their relationship.

Example: Studying the relationship between sales and profit using a scatter plot.

Multivariate Analysis:

Analysis involving three or more variables simultaneously.

Example: Studying relationships between sales, profit, quantity and discount using a pair plot or multivariate visualization.

Therefore:

Univariate → One variable. Bivariate → Two variables. Multivariate → Three or more variables.

QUESTION 3: What insights can be obtained from a correlation heatmap?

A correlation heatmap represents the correlation between numerical variables using colors and correlation values.

It can help identify:

Strong positive relationships. Strong negative relationships. Weak relationships. Highly correlated features. Possible multicollinearity.

Correlation values generally range from -1 to +1.

+1 → Strong positive correlation. -1 → Strong negative correlation. 0 → Little or no linear correlation.

A heatmap therefore provides a quick visual summary of relationships between numerical features.

QUESTION 4: Explain the purpose of histograms, box plots, and scatter plots in EDA.

Histogram:

A histogram shows the distribution of a numerical variable. It helps identify:

Central tendency. Spread. Skewness. Possible multiple peaks.

Box Plot:

A box plot summarizes the distribution of a numerical variable and helps identify possible outliers.

It displays:

Median. Quartiles. Interquartile range. Potential outliers.

Scatter Plot:

A scatter plot shows the relationship between two numerical variables.

It helps identify:

Positive relationships. Negative relationships. Clusters. Outliers. Patterns.

QUESTION 5: How can EDA help identify data quality issues before analysis?

EDA helps identify data quality problems by examining the structure, values and distributions of the dataset.

It can detect:

Missing values using isnull(). Duplicate records using duplicated(). Incorrect data types using info() and dtypes. Outliers using box plots.

Inconsistent categories using value_counts(). Unusual distributions using histograms. Extreme values using descriptive statistics.

Identifying these issues before analysis improves the reliability of the results.

QUESTION 6: Why is correlation important in predictive analytics? Can correlation imply causation?

Correlation is important because it measures the strength and direction of a linear relationship between variables.

In predictive analytics, correlation can help identify potentially useful features and detect highly related variables.

However, correlation does NOT necessarily imply causation.

Two variables can be correlated because:

One variable influences another. Both are influenced by another variable. The relationship occurs by coincidence.

Therefore, correlation indicates association, not necessarily a cause-and-effect relationship.

QUESTION 7: Which visualization would you use to analyze categorical and numerical variables? Justify your choice.

For categorical variables:

Bar charts and count plots are suitable because they show the frequency of different categories clearly.

For numerical variables:

Histograms are useful for understanding the distribution.

Box plots are useful for comparing distributions and identifying outliers.

For relationships between numerical variables:

Scatter plots are useful because they display the relationship between two numerical variables.

Therefore:

Categorical → Bar Chart / Count Plot. Numerical → Histogram / Box Plot. Two Numerical Variables → Scatter Plot.

QUESTION 8: What business insights can be derived from the Netflix (or Superstore/HR Analytics) dataset through EDA?

EDA can provide different business insights depending on the selected dataset.

Netflix Dataset:

Distribution of movies and TV shows. Most common genres. Content distribution by country. Content added over time. Distribution of movie durations.

Superstore Dataset:

Sales performance. Profitability. Product category performance. Regional performance. Relationship between discount and profit.

IBM HR Dataset:

Employee attrition patterns. Job-role distribution. Salary-related patterns. Department-wise employee characteristics. Factors associated with employee attrition.

Therefore, EDA can help organizations identify trends, anomalies and factors that may influence business decisions.

QUESTION 9: How does EDA contribute to feature selection and model building?

EDA helps understand which features may contain useful information for predicting the target variable.

Correlation analysis can identify relationships between numerical features.

Distribution analysis can identify highly skewed variables.

Box plots can identify extreme values.

Categorical analysis can reveal important groups or classes.

EDA can also identify redundant or highly correlated features that may create multicollinearity.

Therefore, EDA helps in:

Selecting useful features. Removing irrelevant features. Detecting redundant features. Choosing suitable preprocessing techniques.

Selecting appropriate machine learning models.

QUESTION 10: What challenges might arise while performing EDA on large-scale real-world datasets?

EDA on large datasets can present several challenges.

High computational requirements: Large datasets require more CPU and memory.

Large storage requirements: Very large datasets may require substantial storage.

Slow visualization: Plotting millions of records can be time-consuming.

Missing and inconsistent data: Large datasets may contain many data quality problems.

High dimensionality: Datasets with many features can be difficult to visualize and analyze.

Processing time: Statistical calculations can take longer on very large datasets.

Sampling, efficient data structures, aggregation and distributed processing can be used to make EDA more manageable on large-scale datasets.